

Models for a variable-sized bin packing problem

Diego Olivier Fernandez Pons

Université Pierre et Marie Curie, Paris VI
 diego-olivier.fernandez-pons@cicrp.jussieu.fr
 ILOG S.A.
 dofpons@ilog.fr

Abstract. Decomposable structure is classically exploited in linear programming with polyhedral based decomposition methods such as Benders decomposition.

We will show how to use the decomposable structure of a variable-sized bin packing problem in constraint programming to derive lower bounds and solve the problem more efficiently.

We will also give evidence that there is still room for improvement, specially in the resolution of sub-problems generated by the decomposition.

1 A variable-sized bin packing problem

In the unsplittable multi-commodity network flow problem with step-increasing cost functions [8] one has to build a network (choose a capacity for every arc of a graph) and assign a path to each demand in such a way that all commodities can flow simultaneously without exceeding the installed capacity (figure 1).

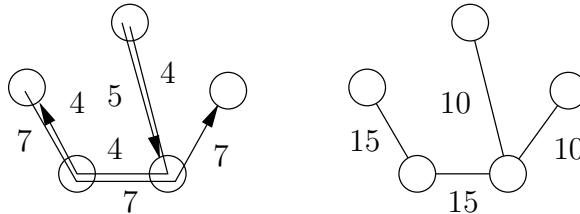


Fig. 1. Paths and arc capacity of a network

Benders decomposition [4], branch and cut [3] and branch cut and price [2] methods use a node-path reformulation of the node-arc linear model that introduces bi-partition inequalities (and more generally metric inequalities). These inequalities state that for any partition $(S, \bar{S})$ of the nodes, the total transversal capacity must be superior to the total transversal flow (figure 2).

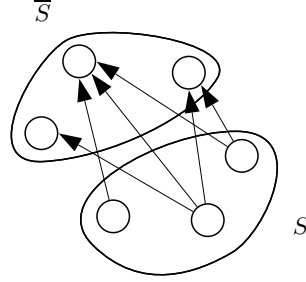


Fig. 2. Demands traversing a cut. The total capacity of the transversal arcs must be superior to the total transversal flow.

$$\forall S, \sum_{i \in S, j \notin S} \text{Demand}_{ij} \leq \sum_{i \in S, j \notin S} \text{Capacity}_{ij}$$

If only the partition $(S, \bar{S})$ is considered and unsplittable-flow constraints added, computing the minimum cost cut such that all the demands can cross the cut is equivalent to the variable-sized bin packing problem: choose a capacity for each bin (arc) and assign a bin (arc) to each item (demand).

Side-constraints on the original network design problem generate several kind of constraints on the packing sub-problem: limiting the flow or the degree on a node (respectively traffic and port with nomult constraints in [8]) become aggregated capacity and cardinality constraints.

$$\begin{aligned} \forall i \in S, \sum_{j \notin S} \text{Capacity}_{ij} &\leq \text{NodeCapacity}_i \\ \forall i \in S, |\{\text{Capacity}_{ij} \neq 0\}| &\leq \text{NodeDegree}_i \end{aligned}$$

We have retained a simpler version that constrains for each bin the total capacity and the number of items it can contain. This variant is known in the CSPLib (<http://4c.ucc.ie/~tw/csplib/>) as “the rack configuration problem” 031.

2 Simple linear and constraint programming models

2.1 A linear formulation with partial symmetry breaking

We begin giving a linear model of the problem:

Let $y_b \in \{0, 1\}^n$ be the indicator vector of the bin type: $y_b = (0, 0, 0, 1, 0, 0, \dots)$ that is $y_b^k = 1$ means bin b has type k .

Let x_b^i be the number of items of type i in bin b

Let M_i be the number of items of type i (constant)

$$\min \sum_b \left(\sum_k \text{Cost}^k y_b^k \right)$$

$$\forall b, \sum_i \text{Size}^i \times x_b^i \leq \left(\sum_k \text{MaxCapacity}^k \times y_b^k \right) \quad (\text{LP } 1)$$

$$\forall b, \sum_i x_b^i \leq \left(\sum_k \text{MaxCardinality}^k \times y_b^k \right) \quad (\text{LP } 2)$$

$$\forall i, \sum_b x_b^i = M_i \quad (\text{LP } 3)$$

$$\forall b, \sum_k y_b^k = 1 \quad (\text{LP } 4)$$

Constraint (LP4) defines the indicator vector y_b , while constraints (LP1) and (LP2) ensure each bin is big enough in terms of capacity and cardinality for the items assigned to it. Constraint (LP3) ensures all items have been assigned to a bin.

Adding lexicographic constraints on the bins ($\forall b, x_b \leq_{lex} x_{b+1}$) to the linear model showed to be less performant than adding an ordering constraint on bin types, even if the latter is only a partial symmetry breaking.

Let $t_b \in \mathbb{N}$ be the type of bin b that is $[y_b^k = 1] \Leftrightarrow [t_b = k]$

$$\forall b, t_b = \sum_k k \times y_b^k \quad (\text{LP } 5)$$

$$\forall b, t_b \leq t_{b+1} \quad (\text{LP } 6)$$

2.2 A constraint programming model

The variable-sized bin packing problem “rack configuration problem” appears in [9] where a simple constraint programming model is suggested.

Let y_b be the type of bin b

Let x_b^i be the number of items of type i in bin b

Let M_i be the number of items of type i (constant).

$$\min \sum_b \text{Cost}[y_b]$$

$$\forall b, \sum_i \text{Size}^i \times x_b^i \leq \text{MaxCapacity}[y_b] \quad (1)$$

$$\forall b, \sum_i x_b^i \leq \text{MaxCardinality}[y_b] \quad (2)$$

$$\forall i, \sum_b x_b^i = M_i \quad (3)$$

Constraints (1), (2) and (3) are similar to those of the linear formulation.

Two symmetry breaking constraints are also introduced: one on bin types, one on the bin content when bins have same type.

$$\forall b, y_b \leq y_{b+1} \quad (4)$$

$$\forall b, [y_b = y_{b+1}] \Rightarrow [x_{0b} \leq x_{0(b+1)}] \quad (5)$$

Kiziltan [7] replaced constraint (5) with a lexicographic ordering of bins with same type.

$$\forall b, [y_b = y_{b+1}] \Rightarrow [x_b \leq_{lex} x_{b+1}] \quad (5 \text{ bis})$$

Both authors first instantiate the bin type and then fill it.

2.3 First experiments

The linear programming model given to ILOG CPLEX 9.1 [5] solves instantaneously all instances of the CSPLib and all instances generated by Kiziltan. On the other hand, the constraint programming model of Van Hentenryck fails to solve half of the CSPLib problems.

We reproduce its results and those reported by Kiziltan on instances of [7] (Kiziltan's experiments were done with ILOG Solver 5.3 on a Pentium III 1Ghz 256Mb of RAM. Ours with a similar configuration at 850MHz).

instance	model	number of fails	time (seconds)
K1	van Hentenryck	3 179 609	151
	Kiziltan $\leq_{lex}$	1 938 252	70
K2	van Hentenryck	5 237 549	258
	Kiziltan $\leq_{lex}$	3 373 548	115
K3	van Hentenryck	2 081 172	110
	Kiziltan $\leq_{lex}$	1 004 283	38
K4	van Hentenryck	2 282 281	124
	Kiziltan $\leq_{lex}$	1 529 197	55
K5	van Hentenryck	15 986 845	811
	Kiziltan $\leq_{lex}$	10 377 867	363
K6	van Hentenryck	199 015	12
	Kiziltan $\leq_{lex}$	115 308	5

Table 1. Simple models results on the rack configuration problem

The analysis of the results shows that for Kiziltan's instances, most of the time is spent in the optimality proof. In the CSPLib problems 3 and 4, the optimal solution still hasn't been found after 1 hour.

3 CP based decomposition

Benders decomposition methods for the network design problem [4] obtain lower bounds by projection [1], that is removing part of the initial problem, here the flow variables. The underlying idea is that the number of potential networks (combination of capacities) is significantly smaller than the number of potential flows (a path in the graph for each demand).

More formally, the problem

$$\min \text{Cost}(y) \mid \exists x, (x, y) \text{ is a feasible network and routing plan}$$

where y correspond to the arc-capacity variables and x to the flow variables is transformed in to a problem on y variables only

$$\min \text{Cost}(y) \mid C_\lambda(y)$$

where C_λ is an exponential set of constraints, namely the metric inequalities. Bi-partition inequalities

$$\forall S, \sum_{i \in S, j \notin S} \text{Demand}_{ij} \leq \sum_{i \in S, j \notin S} \text{Capacity}_{ij}$$

are a particular case of metric inequalities.

Because the number of potential combinations of y is small, just enforcing a subset of the constraints C_λ may be enough to find good lower bounds efficiently.

3.1 Projection of the packing problem

We mimic Benders decomposition considering the problem restricted to capacity variables y only and the following aggregated inequalities (we keep notations of section 2.2)

$$\min \sum_b \text{Cost}[y_b]$$

$$\sum_i \text{Size}_i \times M_i \leq \sum_b \text{MaxCapacity}[y_b] \quad (\pi \ 1)$$

$$\sum_i M_i \leq \sum_b \text{MaxCardinality}[y_b] \quad (\pi \ 2)$$

$$\forall b, y_b \leq y_{b+1} \quad (4)$$

Constraints $(\pi 1)$ and $(\pi 2)$ are obtained injecting (3) in (1) and (2) and summing over b . In the process x_b^i disappears which means we relax its integrality. In other words any solution of the projected model is optimal if items are allowed to split among several bins.

Lower bounds were tested on CSPLib and Kiziltan instances (table 2). K 5 is an interesting case because it has the longest resolution time with constraint programming naive models while there is a single bin combination that may be feasible. These problems are examples of pairs (x, y) where the number of candidate y is very small compared to the size of x .

instance	lower bound	table size	choice points	fails	seconds
CSPLib 1	550	13	13	1	0
CSPLib 2	1100	40	40	1	0.01
CSPLib 3	1200	61	60	0	0
CSPLib 4	1150	3486	4104	619	0.15
K 1	950	2	2	1	0
K 2	950	2	2	1	0
K 3	∞	0	0	1	0
K 4	900	3	3	1	0
K 5	1000	1	0	0	0.01
K 6	∞	0	0	1	0

Table 2. Projection lower bounds for CSPLib and Kiziltan instances

Comparing lower bounds with CPLEX root node and optimal solutions show that bounds are exact for CSPLib and Kiziltan problems.

instance	CPLEX root	lower bound	optimal
CSPLib 1	535	550	550
CSPLib 2	1070	1100	1100
CSPLib 3	1170	1200	1200
CSPLib 4	1140	1150	1150
K 1	940	950	950
K 2	930	950	950
K 4	900	900	900
K 5	1000	1000	1000

Table 3. Lower bounds compared to CPLEX root and optimal solutions

A small instance named **easy** (figure 3) has been designed in such a way there is a gap between the lower bound of the splittable item relaxation and the optimal solution. **easy** $\times 2$ means the number of bins and the number items of each size has been doubled. We compare (table 4) the value of CPLEX root node, the lower bound and the optimal solution. The gap indicates the number of solutions of the projection problem that lie between the lower bound and the optimal value (both included).

instance	CPLEX root	lower bound	optimal	gap
easy	4 230	4 500	4 625	3
easy×2	8 640	8 750	9 125	4
easy×4	16 920	17 375	17 875	6
easy×8	33 840	32 025	35 400	27
easy×16	67 680	67 950	70 075	112
easy×32	135 360	135 650	140 150	946

Table 4. Lower bounds compared to CPLEX root and optimal solutions

```

# bin types (capacity, cardinality, cost)
200 4 125
150 8 150
200 6 500
250 6 600

# item types (size, number)
20 15
30 8
40 15
50 10
60 10

# total number of bins
10

```

Fig. 3. easy instance used in our experiments

3.2 Tighter lower bounds

The covering projection problem can be strengthened introducing waste variables

$$\sum_b \text{MaxCapacity}[y_b] \geq \left(\sum_i \text{Size}_i \times M_i \right) \quad (\pi \ 1)$$

$$\sum_b \text{MaxCardinality}[y_b] \geq \left(\sum_i M_i \right) \quad (\pi \ 2)$$

become

$$\sum_b \text{MaxCapacity}[y_b] = \left(\sum_i \text{Size}_i \times M_i \right) + \sum_b \text{WastedCapacity}_b \quad (\sigma \ 1)$$

$$\sum_b \text{MaxCardinality}[y_b] = \left(\sum_i M_i \right) + \sum_b \text{WastedCardinality}_b \quad (\sigma \ 2)$$

where the wasted variables are linked to bin capacities by the relations

$$\begin{aligned}\forall b, \left(\sum_i \text{Size}^i \times x_b^i \right) + \text{WastedCapacity}_b &= \text{MaxCapacity}[y_b] \\ \forall b, \left(\sum_i x_b^i \right) + \text{WastedCardinality}_b &= \text{MaxCardinality}[y_b]\end{aligned}$$

To make disappear variables x_b^i we enumerate them, in other words we consider all possible packings of a bin (that is all x^i vectors) and compute every feasible tuple $(y_b, \text{WastedCapacity}_b, \text{WastedCardinality}_b)$.

For the **easy** family we obtain 223 tuples which can be reduced to 77 using a *domination rule*: only bins of minimal are kept because there is no point in using a bin more expensive than a bin where x^i already fits.

With ILOG Solver[6], variables can be constrained to take their values in a tuple list with a table constraint. To be able to compute strengthened lower bounds also with a linear solver, tuples are expanded: to each tuple $t = (\text{type}, \text{cap}, \text{card})$ is associated a variable n_t which counts the number of times a bin of type **type** with wasted capacity **cap** and wasted cardinality **card** appears in the optimal solution.

The problem becomes

$$\min \sum_t \text{Cost}[n_t]$$

$$\sum_t (\text{MaxCapacity}[\text{type}_t] - \text{cap}_t) \times n_t = \sum_i \text{Size}_i \times M_i \quad (\sigma \text{ LP } 1)$$

$$\sum_t (\text{MaxCardinality}[\text{type}_t] - \text{card}_t) \times n_t = \sum_i M_i \quad (\sigma \text{ LP } 2)$$

$$\sum_t n_t = \text{NumberOfBins} \quad (\sigma \text{ LP } 3)$$

For instances **easy** to **easy**×32, the strengthened lower bound reach the optimal solution of the problem.

4 Two phases constraint programming models

It is interesting to notice that just introducing waste variables in (1) and (2), their summations $(\sigma 1), (\sigma 2)$ and changing slightly the branching scheme (all bin types are instantiated and then they are filled) allows to solve all Kiziltan problems instantaneously. Adding the table of discrete wasted values allows to solve all CSPLib instances and a few **easy** instances (table 5).

$$\min \sum_b \text{Cost}[y_b]$$

$$\forall b, \left(\sum_i \text{Size}^i \times x_b^i \right) + \text{WastedCapacity}_b = \text{MaxCapacity}[y_b] \quad (1 \text{ bis})$$

$$\forall b, \left(\sum_i x_b^i \right) + \text{WastedCardinality}_b = \text{MaxCardinality}[y_b] \quad (2 \text{ bis})$$

$$\sum_b \text{MaxCapacity}[y_b] = \left(\sum_i \text{Size}_i \times M_i \right) + \sum_b \text{WastedCapacity}_b \quad (\sigma 1)$$

$$\sum_b \text{MaxCardinality}[y_b] = \left(\sum_i M_i \right) + \sum_b \text{WastedCardinality}_b \quad (\sigma 2)$$

$$\forall i, \sum_b x_b^i = M_i \quad (3)$$

$$\forall b, y_b \leq y_{b+1} \quad (4)$$

$$\forall b, [y_b = y_{b+1}] \Rightarrow [x_b \leq_{lex} x_{b+1}] \quad (5 \text{ bis})$$

$$\forall b, (y_b, \text{WastedCapacity}_b, \text{WastedCardinality}_b) \in \text{table} \quad (6)$$

Constraints $(\sigma 1)$ and $(\sigma 2)$ prevent trivially infeasible combinations of bins to be generated (e.g. all the bins assigned to the empty type). Then, the total wasted capacity, the total wasted cardinality (fixed) and the table of discrete wasted values constrain the waste values of individual bins, which ends imposing a minimum load to some bins and reduces the number of candidate x^i vectors. Finally, because bin types are fixed, the lexicographic symmetry breaking constraint can prune efficiently.

The CP model tends however to get stuck in a bin combination without being able to fill it or prove it infeasible. This suggests that the fixed-size bin packing problem is not properly solved. Putting all bin possible vectors x^i in a table induced a small overhead, therefore was retained to help the filling phase.

Finally, these first results suggest that a two phases approach may capture in a better way the structure of the problem.

instance	easy	easy $\times 2$	easy $\times 4$	easy $\times 8$
Solver 6.1 waste	0.24	0.29	-	-
Solver 6.1 waste + table	0.17	0.28	116.79	-
CPLEX 9.1 balance	0.75	61.07	(19925)	-(47050)
CPLEX 9.1 integer	0.54	(9125)	-	-
CPLEX 9.1 optimality	0.84	6.94	(21200)	-
CPLEX 9.1 best bound	1.95	9.57	-	-
CPLEX 9.1 hidden	1.23	(9125)	-(17975)	-

Table 5. Time is in seconds, '-' means the solver was stopped after 2 hours, the best solution found if any is given. CPLEX was tried with all of its MIP Emphasis parameters. Experiments were done on a Pentium III 500Mhz 256Mb of RAM

4.1 A descending approach

The idea of descending approaches is to use the lower bound argument at every node of the search tree. Instead of computing an optimal solution of projected problems of section 3.1 or 3.2 at every node, we enumerate all its solutions at the root node and store them in a table.

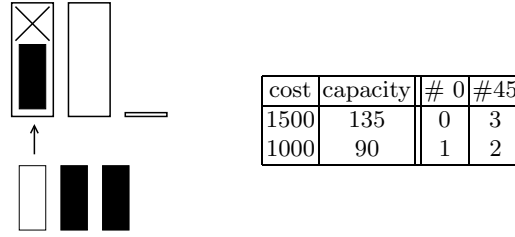


Fig. 4. An example of lower bound raising after a branching

Figure 4 shows how the lower bound may raise while the bins are filled: 3 items of size 30 have to be packed and the only available bins are empty bins or bins of capacity 45 and cost 500. The table contains all combinations of 3 bins such that their total capacity is larger than 90. At the root node (no item has been packed), the minimal cost configuration (45, 45, 0) is still possible. When the first item is assigned to the first bin, a wasted capacity of $15 = 45 - 30$ is computed. The total wasted capacity raises from 0 to 15 and the total capacity lower bound raises from $90 = 3 \times 30$ to $105 = 3 \times 30 + 15$ (equation $\sigma 1$). The table then increases the lower bound on the total cost to 1500 because it is the minimal cost of a bin combination with a capacity larger than 105 (3 bins of size 45).

The descending approach results appear in table 6 as 'descent table'.

4.2 An ascending approach

Ascending approaches fix the bin combination of lowest cost and try to fill it. If it is proved unfeasible, the second best bin combination is tried and so on.

The ascending scheme was first implemented enumerating all solutions of the projection problems in section 3.1 (ascent table) and 3.2 (ascent table strong), and storing them in a table. The simple lower bound table is larger than the strong one, but faster to compute. Tables become really large when the size of the problem increases: for **easy**×16, the simple lower bound table contains 121 000 tuples of arity 160 and the strong lower bound table 100 000 tuples.

A second implementation labels bin types in a best first search way (using the cost of the combination which only depends on y_b as evaluator) allowing to circumvent the very high memory consumption of table based implementations. Candidate configurations are guaranteed to be visited in increasing order of cost. Only the simple lower bound is used because the strong model results in higher computation times. Results appear as 'ascent bfs' in table 6.

Table 6 shows the results of two phases descending or ascending methods on the **easy** problems.

instance	easy	easy ×2	easy ×4	easy ×8	easy ×16	easy ×32
Solver 6.1 descent table	0	2	(18800)	-		
Solver 6.1 ascent table	0	0	2	15	-	
Solver 6.1 ascent table strong	0	1	4	93	-	
Solver 6.1 ascent bfs	0	0	1	17	270	6076

Table 6. Time is in seconds, '-' means the solver was killed by the OS because of too high memory consumption, the best solution found if any is given. An empty space means the test wasn't performed. Experiments were done on a Pentium III 850Mhz 256Mb of RAM

5 Hard sub-problems

For both descending and ascending methods, the dominating time in **easy** instances is the lower bound computation. We built a family of problems named **medium** for which lower bounds are poor, hence most of the time is spend trying to fill an infeasible combination of bins. The ascending solvers end finding a level where they get trapped.

medium problems are easier than **easy** ones for linear programming methods. Linear solvers seem to have be in more trouble when the problem is very discrete

```

# bin types (capacity, cardinality, cost)
200 4 125
150 8 150
400 6 250
300 8 300
200 15 500
400 8 600
400 10 900
500 10 1000

# item types (size, number)
20 15
30 8
40 15
50 10
60 10

# total number of bins
10

```

Fig. 5. medium instance used in our experiments

instance	CPLEX root	lower bound	gap	strong lower bound	gap	optimal
medium	1 595	1 700	25	1 725	5	1 875
medium×2	3 190	3 300	209	3 540	33	3 675
medium×4	6 380	6 475	3 188	6 900	244	7 275
medium×8	12 760	12 825	103 331	13 700	3 026	14 475

Table 7. Lower bounds compared to CPLEX root and optimal solutions

(e.g. when bin sizes are small with respect to item sizes), whereas decomposition-based constraint programming approaches fail when the problems tends to be continuous.

instance	medium	medium ×2	medium ×4	medium ×8
Solver 6.1 ascent bfs	1.48	-		
CPLEX 9.1 balance	0.76	8.48	65.82	(28250) -
CPLEX 9.1 integer	0.4	4.57	375.62	7262.87
CPLEX 9.1 optimality	0.58	12.49	98.52	(14600) -
CPLEX 9.1 best bound	2.34	22.24	147.99	3622.41
CPLEX 9.1 hidden	0.7	11.41	126.77	(14550) -

Table 8. Time is in seconds, '-' means the solver was stopped after 2 hours, the best solution found if any is given. CPLEX was tried with all of its MIP Emphasis parameters. Experiments were done on a Pentium III 850Mhz 256Mb of RAM

While computing lower bounds for both **easy** and **medium** problems is almost instantaneous with the linear and combined linear-CP models described in sections 3.1 and 3.2, their corresponding constraint programming models are quite slow. Since the covering problems that appear in lower bounds computations are common, a large number of applications would benefit from significant improvements in this direction.

```
# bin types (capacity, cardinality, cost)
200  4  125
150  8  150
400  6  250
300  8  300
200 15  500
400  8  600
400 10  900
500 10 1000

# item types (size, number)
20  15
30  8
40  15
50  10
60  10
70  20
80  3
90  5
100 12
110 9

# total number of bins
24
```

Fig. 6. **hard** instance

On **hard** instance, linear programming solvers find the optimal value soon but aren't able to prove it. Our constraint programming approaches don't return any solution.

6 Conclusion

Projection and decomposition techniques usually used in linear programming to obtain lower bounds can also be useful in constraint programming based methods.

We showed on the variable-sized bin packing problem (the “rack configuration problem” 031 in the CSPLib) how one could obtain lower bounds, strengthen

them with discrete arguments and integrate them into several kinds of constraint programming approaches that improve previous models.

We also gave efficient linear programs and combined linear CP programs to compute these lower bounds, highlighting directions where significant improvements are desirable.

References

1. Egon Balas. *Computational Combinatorial Optimization: Projection and Lifting in Combinatorial Optimization*, pages 26–56. Springer-Verlag, 2001.
2. Alain Chabrier, Emilie Danna, Claude Le Pape, and Laurent Perron. Solving a network design problem. *Annals of Operations Research*, 130(1-4):217–239, 2004.
3. Mervat Chouman, Teodor Gabriel Crainic, and Bernard Gendron. A cutting-plane algorithm based on cutset inequalities for multicommodity capacitated fixed charge network design. Technical Report CRT-2003-16, Centre de recherche sur les transports, Universit  de Montral, 2003.
4. Virginie Gabrel, Arnaud Knippel, and Michel Minoux. Exact solution of multicommodity network optimization problem with general step cost functions. *Operations Research Letters*, 25:15–23, 1999.
5. ILOG. *ILOG Cplex 9.1 User Manual*. ILOG, april 2005.
6. ILOG. *ILOG Solver 6.1 User Manual*. ILOG, april 2005.
7. Zeynep Kiziltan. *Symmetry breaking ordering constraints*. PhD thesis, Uppsala University, 2004.
8. Claude Le Pape, Laurent Perron, Jean-Charles R gin, and Paul Shaw. Robust and parallel solving of a network design problem. In *8th International conference on principles and practice of constraint programming (CP)*, pages 633–648, 2002.
9. Pascal Van Hentenryck. *The OPL optimization language*. The MIT Press, 1999.